

This document defines the High-Level Design for a production-ready AI Platform. The stack is intentionally pragmatic — starting lean with battle-tested tools, then evolving toward advanced capabilities in clearly defined phases.

PostgreSQL as Source of Truth	All structured data lives here. ACID-compliant, well-understood, horizontally scalable via read replicas.
pgvector for AI Memory & RAG	Vector similarity search co-located with relational data. No separate vector DB needed in early phases.
OpenRouter for Model Abstraction	Single API surface to GPT, Claude, Gemini, Groq, DeepSeek. Enables fallback and cost routing.
S3 for All Blob Storage	PDFs, images, videos, exports, logs. Never store large files in PostgreSQL.

Auth Service

Feature	Description
Users & Profiles	Registration, login, OAuth, API Keys
Sessions	JWT + refresh token management
OAuth Accounts	Google, GitHub, Microsoft SSO

Chat Service

Feature	Description
Conversations	Full persistence, no LLM vendor lock-in
Messages	All turns stored with metadata & cost
Attachments	Linked to S3 objects via message_attachments

Model Router

Feature	Description
Manual Selection	User explicitly picks Claude, GPT, etc.
Auto Fallback	Claude fails → GPT → Gemini → DeepSeek
Cost Routing	Route to cheapest model meeting quality bar
Latency Routing	Route to fastest model for real-time needs

Notes Service

Feature	Description
note_blocks	Block-based editor (like Notion)
note_tags	Tagging and categorisation
note_embeddings	Semantic search across all notes

3

MEMORY ARCHITECTURE

Memory is structured in four distinct layers, each serving a different temporal and semantic purpose.

	Short-Term Context Last 30–50 messages held in the active context window. Managed by the Chat Orchestrator.	• <code>messages</code> table • Recent turns only • Fast retrieval
	User Memory Extracted facts about the user stored in <code>user_memories</code> with confidence scores.	• <code>user_id</code>, <code>memory</code>, <code>confidence_score</code> • e.g. 'User prefers index investing' • Persistently updated
	Semantic Memory Embeddings stored in <code>pgvector</code> . Enables retrieval of conversations from months ago.	• <code>memory_embeddings</code> table • cosine similarity search • ANN via <code>pgvector</code> HNSW
	Knowledge Memory Generated from Notes, Documents and Integrations. The richest long-term context.	• Notes + Documents + Integrations • RAG-powered retrieval • Re-ranked before injection

4

RAG ARCHITECTURE

The Retrieval-Augmented Generation pipeline converts uploaded documents into semantically searchable knowledge, injected into the LLM context at query time.

1. Upload PDF User uploads document via frontend	2. Store in S3 Raw file saved, metadata recorded in <code>documents</code> table	3. Chunking Document split into overlapping chunks (~512 tokens)	4. Embeddings Each chunk embedded via OpenAI <code>text-embedding-3-small</code>	5. pgvector Store Vectors stored in <code>document_embeddings</code> with <code>chunk_id</code> FK
6. Retrieval Query embedded → top-K chunks via ANN search	7. Reranking Cohere or cross-encoder reranker refines results	8. Prompt Assembly Top chunks injected into system prompt	9. LLM Final answer generated with full context	

PostgreSQL (Source of Truth)

Table / Key	Purpose
users, profiles	Core identity and settings
conversations, messages	Full chat history
user_memories	Extracted memory facts with confidence
notes, note_tags, note_blocks	Knowledge hub
documents, document_chunks	RAG source material
integrations	OAuth tokens and sync state
agents, agent_tools	Future-ready agent registry
audit_logs, usage_logs	Compliance and analytics

Redis (Ephemeral Layer)

Table / Key	Purpose
Session store	JWT refresh tokens, fast auth
Rate limiting	Per-user, per-model token buckets
Streaming buffer	SSE chunks for live LLM responses
Job queue	Background tasks (embed, sync, export)
Response cache	Short-TTL cache for identical queries

S3 (Object Storage)

Table / Key	Purpose
PDFs & documents	Source files for RAG pipeline
Images & videos	User-uploaded attachments
Exports	Generated reports and note exports
Logs & backups	Audit trails, DB snapshots

Web Search Layer

A provider-agnostic search interface abstracts over Tavily, Serper, and Google Search API. Search results are injected into context before LLM calls when the query requires current information.

Provider	Strength	Use Case
Tavily	AI-optimised	Best for factual Q&A; and research
Serper	Low cost	High-volume, cost-sensitive workloads
Google Search API	Authoritative	News and authoritative sources
Custom Crawlers	Full control	Internal or restricted content

Integrations Layer (OAuth)

Platform	Data Available	Storage
Gmail	Emails, threads, labels, attachments	integration_accounts + tokens
Slack	Messages, channels, DMs, files	integration_sync_jobs
Notion	Pages, databases, blocks	Incremental sync via webhooks
Google Drive	Docs, Sheets, PDFs	Metadata in DB, files in S3
Google Calendar	Events, attendees, availability	Cached in Redis

BUILD ORDER & PHASES

Six clearly scoped phases let the team ship value early and evolve without redesigning the foundation.

PHASE 1

MVP

- React + FastAPI
- PostgreSQL + pgvector
- OpenRouter model switching
- Conversations & messages

Target: ChatGPT clone with model switching

PHASE 2

Memory

- User memory extraction
- Semantic search (pgvector)
- Notes system

Target: Personal AI Memory System

PHASE 3

Knowledge

- PDF upload + chunking
- RAG pipeline
- Knowledge base

Target: Perplexity + Notion style

PHASE 4

Integrations

- Gmail, Slack, Notion
- Google Drive + Calendar
- OAuth layer

Target: Personal AI Workspace

PHASE 5

Agents

- Research agent
- Coding agent
- Email + Calendar agents
- Workflow automation

Target: AI Operating System

PHASE 6

SaaS

- Analytics dashboard
- Cost management
- Multi-tenant support
- Enterprise features

Target: Commercial SaaS Platform

ANALYTICS & OBSERVABILITY

Event / Metric	Table	Purpose
Token usage per model	model_usage	Cost attribution and budgeting
Provider cost tracking	cost_tracking	ROI analysis, cheapest model routing
Latency per request	usage_events	SLA monitoring, model comparison
Active users (DAU/WAU/MAU)	user_activity	Growth and retention metrics
Search query volume	search_usage	Search provider optimisation
Knowledge base queries	usage_events	Feature engagement analytics
Agent task completions	usage_events	Agent quality and success rate

RECOMMENDED FOLDER STRUCTURE

```

apps/
  ■■■ frontend/      React application
  ■■■ backend/       FastAPI application

services/
  ■■■ auth/          User registration, OAuth, sessions
  ■■■ chat/           Conversation & message persistence
  ■■■ memory/         User memory extraction + embeddings
  ■■■ notes/          Note blocks, tags, embeddings
  ■■■ rag/            Document chunking + retrieval
  ■■■ search/         Web search provider abstraction
  ■■■ integrations/   Gmail, Slack, Notion, Drive
  ■■■ analytics/      Usage events + cost tracking
  ■■■ agents/         Future-ready agent registry

packages/
  ■■■ models/         Shared data models / types
  ■■■ prompts/        Prompt templates and builders
  ■■■ embeddings/     Embedding helpers and adapters
  ■■■ utilities/      Shared utils, validators, parsers

infrastructure/
  ■■■ postgres/       Schema migrations (Alembic)
  ■■■ redis/           Config and TTL policies
  ■■■ docker/          Compose files for local dev
  ■■■ kubernetes/     Production deployment manifests

```

FINAL RECOMMENDATION

Your strongest architecture is built on a simple, well-understood foundation. Start here and evolve — never redesign the foundation.

React
Frontend

FastAPI
API Layer

PostgreSQL
Source of Truth

pgvector
AI Memory &
RAG

Redis
Cache & Queue

S3
Object Storage

OpenRouter
Model Router

Build the platform around **Chat + Memory + Notes + Knowledge Base** first. Everything else — agents, analytics, advanced integrations — can be layered on top without needing to redesign the foundation. PostgreSQL is not a bottleneck until you are at serious scale; do not introduce Snowflake or a data lake prematurely.